TASK-8

KNAPSACK PROBLEM USING DYNAMIC PROGRAMMING

Problem You are given n objects, a knapsack of capacity c, array v, and array w. The ith object has value V[i] and weight w[i].
Determine the maximum total value that you can get by selecting objects in such a manner that their sum of weights is not greater than the capacity c.

Hint : Use Dynamic programming

Input Format : First line : Two integers n and c denoting the number of objects and capacity of the knapsa-ck ($1<=n<=10^3$ and $1<=c<=2*10^6$)
Second line : n integers ($0<=Vi<=50$)
Third line : n integers ($10<=wi<=2*10^6$)

Output Format : Print a single integer denoting the maximum value that you can get by selecting the objects.

Sample input :
4 20
10 2 1 3
10 5 10 10

Sample output : 13

ALGORITHM : 1. Create a 2D array : intialize a 2D array dp of size $(n+1)x(W+1)$ with all values set to 0.
2. Iterate through items and capacities :
- For each item i from 1 to n :
- For each capacity j from 1 to w :
  - If the weight of the current item (wt[i-1]) is greater than current capacity (j):
    - dp[i][j]= dp[i-1][j] // Don't include current ite
    - otherwise

PROGRAM:

```
#include<stdio.h>
# include<string.h>
#include <limits.h>
# define MAX_N 1000
# define MAX_VAL_PER_ITEM50
# define MAX_TOTAL_VAL (MAX_N* MAX_VAL_PER_ITEM)
long long min_weight [MAX_TOTAL_VAL+1];
int main(){
  int n, c;
  Scanf ("%d %d", &n, &c);
 int v[n], w[n];
 for (int i=0; i<n; i++){
   Scanf ("%d", &v[i]);
 }
  for (int i=0; i<n; i++){
  Scanf ("%d", &w[i]);
 }
// Calculate maximum possible value
   int max_possible_val = 0;
   for (int i=0; i<n; i++){
     max_Possible_Val + = v[i];
  }
// Intialize DP array
 for (int i=1; i<= max_possible_val; i++){
   min_weight [i] = LLONG_MAX;
  }
   min weight [0] = 0;
//DP : for each item, update minimum weight for each
     Possible value
 for (int i=0; i<n; i++){
  for (int val = max_possible_val; val >= v[i]; val--) {
```

```
        if (min_weight[val-V[i]] ! = LLONG_MAX){
        Conglong new_weight = min_weight[val-V[i]]+w[i];
        if (new_weight < min_weight[val]){
        min_weight[val] = new_weight;
        }
      }
    }
  }

  // Find maximum value that fits in knapsack
        int result=0;
      for (int val = max_possible_val; val >=0 ; val--){
        if (min_weight [val]<=c) {
          result = val;
          break;
        }
      }
      Printf ("%d\n", result);
      return 0;
  }
```

VELTECH	
EX.No.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	

RESULT: Thus the task was sucessfully executed and verified sucessfully.